

★ Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)



Demonstration of Hyperledger Aries Cloud Agent

10 min read · Dec 9, 2019



KC Tam

Follow



Listen



Share



More

Overview

Since Hyperledger Aries was born in March, we continue seeing progress in the development. One implementation is the [Aries Cloud Agent in Python](#). Though the development is still going on, some demos are provided, and among them, the [OpenAPI demo](#) gives us a better picture on how we interact with the cloud agent through APIs. This is the foundation to build application using the Aries cloud agent.

The [description](#) has already given us a very complete picture. Again, I write this article as an illustrative guide on this demo. With screen shots and explanation I hope you again grab some ideas on the agent itself, and from the OpenAPI we can understand how we can develop application on top of it.

Quick Introduction on Hyperledger Aries

Before the separation, the agent software is part of Hyperledger Indy. Since this March, the agent is taken out and accepted as a new project. Here is how Hyperledger Aries is created. Hyperledger Aries is the toolkit focusing on creating, transmitting, storing and verifying digital credentials. The aim of the separation comes with the vision that the agent software can be used on multiple ledger platforms (though today it only works with Indy ledger).

Here you can find the repository of [Hyperledger Aries](#).

Quick Note about Swagger OpenAPI

The Aries Cloud Agent - Python comes with a very handy OpenAPI (Swagger) interface. This provides the REST API interface when one develops controller interacting with the agent. Besides, OpenAPI also provides a nice user interface during development and demonstration.

The article does not provide the detail on the interface. In short, locate the right API (either GET or POST), press Try it out. Some API requires input the argument or the body, and use Execute to execute the API. Observe the Response for status or some useful information returned.

Refer [here](#) for more about OpenAPI used in Aries Cloud Agent - Python.

Demo Overview

I begin with the repository, but have made some modification. Therefore the [code](#) I am using in my GitHub repository, which is forked from the original code. Certain parts are added or modified. The change I have made is

- comment out the invitation from Faber and Acme Agent (the invitation is done on OpenAPI)
- change an attribute “age” to “average”, and the proof request is average greater or equal to 4
- create a Bob agent, with the code copied and modified from Alice, the port is changed to 8050

The storyline is almost kept the same, except that we will have Bob now.

- Organizations in this demo: Faber College and Acme Corp
- Both Alice and Bob graduate from Faber College.
- Acme Corp has a job opening, requiring an education qualification.
- Alice and Bob apply Transcript from Faber College, which is in the format of credential.
- Alice and Bob submits proof (presentation) to Acme Corp in order to meet the education requirement.

In implementation, there are four agents in this demo. Faber, Acme, Alice and Bob. All are running aca-py cloud agent, and during the demo we are using the OpenAPI

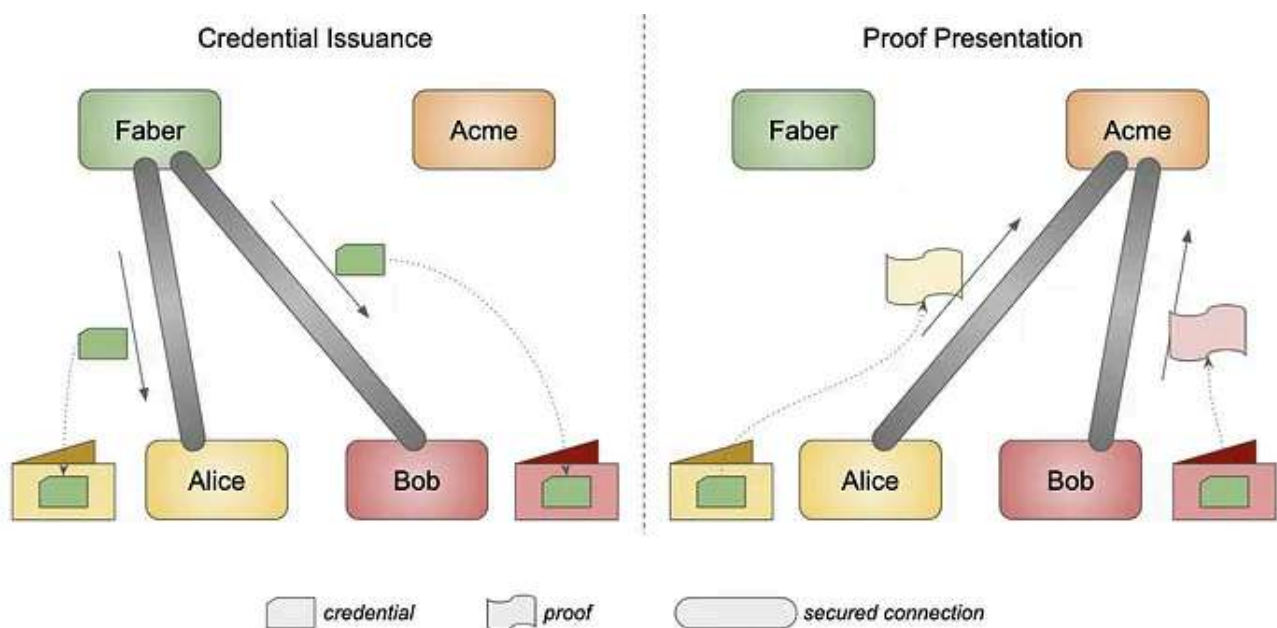
to POST / GET their cloud agents. Each agent is implemented as a container in a local host.

Besides, we also need a ledger to store relevant information. In this demo we are using von-network, which implements an Indy ledger with four nodes. The von-network comes with pre-defined stewardship setup, and we simply use them.

We will walk through the demo in this flow.

1. Preparation of Demo Setup
2. Credential Issuance by Faber to Alice and Bob
3. Proof presentation by Alice and Bob to Acme upon Acme's Proof Request

This diagram summaries the interaction of various agents in credential issuance and proof presentation.



Demo of credential issuance and proof presentation using Aries Cloud Agent - Python

Step 1: Preparation of Demo Setup

Open a terminal to bring up von-network.

```
git clone https://github.com/bcgov/von-network
cd von-network
./manage up
```

When the network is up and running we can open a browser and access the ledger by `http://localhost:9000`.

Then we bring out another four terminals, one for each cloud agent.

```
git clone https://github.com/kctam/acapy-alice-bob
cd acapy-alice-bob/demo

# on each terminal
./run_demo faber
./run_demo acme
./run_demo alice
./run_demo bob
```

Something is predefined on this demo

- The Faber agent registers a public DID into the ledger, and creates a Schema “Degree Schema” and a Credential Definition based on this Schema
- The Acme agent registers a public DID into the ledger
- The public DIDs of Faber and Acme are written into the ledger. So are the schema and credential definition.
- Both Alice and Bob agent do not register a public DID

We are using OpenAPI to send instruction to each agent. The port predefined in the client code is

- Faber: 8021
- Acme: 8041
- Alice: 8031
- Bob: 8051

Open browser using `http://localhost:<port>` for each agent in order to access the OpenAPI. With these four browsers, we are ready for the demonstration.

Obtain the DIDs of both Faber and Acme, and the schema and credential definition from the agent terminals.

```
#1 Provision an agent and wallet, get back configuration details
Faber | Registering Faber Agent with seed_d_00000000000000000000993247
Faber | Get DID: VA3GvaAZRZwrH4dDtVW6E
Faber | 
Faber | ::::::::::::::::::::::::::::
Faber | :: Faber Agent ::
Faber | :: ::
Faber | :: ::
Faber | :: Inbound Transports: ::
Faber | :: ::
Faber | :: - http://0.0.0.0:8020 ::
Faber | :: ::
Faber | :: Outbound Transports: ::
Faber | :: ::
Faber | :: - http ::
Faber | :: - https ::
Faber | :: ::
Faber | :: Public DID Information: ::
Faber | :: ::
Faber | :: - DID: VA3GvaAZRZwrH4dDtVW6E ::
Faber | :: ::
Faber | :: Administration API: ::
Faber | :: ::
Faber | :: - http://0.0.0.0:8021 ::
Faber | :: ::
Faber | :: ver: 0.3.5 ::
Faber | ::::::::::::::::::::::::::::
Faber | 
Faber | Listening...
Faber | 

Startup duration: 9.03s
Admin url is at: http://192.168.65.3:8021
Endpoint url is at: http://192.168.65.3:8020

#3/4 Create a new schema/cred def on the ledger
Schema ID: VA3GvaAZRZwrH4dDtVW6E:2;degree:schema:70.19.14
Cred def ID: VA3GvaAZRZwrH4dDtVW6E:3;CL:8;default
```

```
#1 Provision an agent and wallet, get back configuration details
Acme | Registering Acme Agent with seed d_00000000000000000000778136
Acme | Got DID: 6K4zPHKFCQvXoACos7mMb
Acme |
Acme | ::::::::::::::::::::::::::::::::::::
Acme | :: Acme Agent ::
Acme | :: ::
Acme | :: ::
Acme | :: Inbound Transports: ::
Acme | :: ::
Acme | :: - http://0.0.0.0:8040 ::
Acme | :: ::
Acme | :: Outbound Transports: ::
Acme | :: ::
Acme | :: - http ::
Acme | :: - https ::
Acme | ::
Acme | :: Public DID Information: ::
Acme | :: ::
Acme | :: - DID: 6K4zPHKFCQvXoACos7mMb ::
Acme | :: ::
Acme | :: Administration API: ::
Acme | :: ::
Acme | :: - http://0.0.0.0:8041 ::
Acme | :: ::
Acme | :: ver: 0.3.5 ::
Acme | ::::::::::::::::::::::::::::::::::::
Acme |
Acme | Listening...
Acme |
Startup duration: 13.27s
Admin url is at: http://192.168.65.3:8041
Endpoint url is at: http://192.168.65.3:8040
```

Agent terminals for Faber and Acme, where we see their public DIDs, and the schema and credential definition.

For **record purpose**, here is the record of my setup. You will have your own value.

- Faber Public DID: *VA3GvaAZRZwrH4dDtVWd6E*
- Acme Public DID: *6K4zPhKFCQvXaACos7mvMb*
- Schema ID of “degree schema”: *VA3GvaAZRZwrH4dDtVWd6E:2:degree schema:70.19.14*
- Credential Definition ID for “degree schema”:
VA3GvaAZRZwrH4dDtVWd6E:3:CL:8:default

The schema Faber creates has the following information

- name: “degree schema”
- version: x.y.z (all are randomly generated)
- attributes: name, date, degree and average

Step 2: Credential Issuance

Before any communication happens, a secured connection is established between two agents. In this demo, Faber first establishes one connection to Alice, and one to Bob. After that Faber issues credential to them with their own personal data. Finally Alice and Bob will see the credential in their own wallets.

Step 2.1: Faber establishes connections to Alice and Bob

First, Faber establishes a connection to Alice.

- Faber Agent: POST /connections/create-invitation , from the response get the invitation object (from { to })
- Alice Agent: POST /connections/receive-invitation with the invitation object
- Check both Faber Agent and Alice Agent by GET /connections and both are in **Active** status

The screenshot illustrates the process of creating a connection between Faber and Alice. It shows two API endpoints and their respective responses and request bodies.

Alice's Cloud Agent API
 /connections/receive-invitation

Faber's Cloud Agent API
 /connections/create-invitation

The Faber API response (200) shows the invitation object:

```
{
  "connection_id": "51a5da3-749e-44d9-8a21-c51040b43221",
  "invitation": {
    "type": "did:sov:R3CkNTHM3181G2TUAR8Q:spec/connections/1.0/invitation",
    "id": "02ec12d1-9549-42b2-9f66-4a899291b6b7",
    "serviceEndpoint": "http://192.168.65.3:8020",
    "label": "Faber Agent",
    "recipientKeys": [
      "G5SC6Z3Y28GLU35A80097A8A5311eN8E7r0u95338e"
    ]
  }
}
```

The Alice API request body (copy invitation and paste here) shows the invitation object:

```
{
  "type": "did:sov:R3CkNTHM3181G2TUAR8Q:spec/connections/1.0/invitation",
  "id": "02ec12d1-9549-42b2-9f66-4a899291b6b7",
  "serviceEndpoint": "http://192.168.65.3:8020",
  "label": "Faber Agent",
  "recipientKeys": [
    "G5SC6Z3Y28GLU35A80097A8A5311eN8E7r0u95338e"
  ]
}
```

As an example, Faber establishes a connection to Alice.

Similar for connection between Faber to Bob.

- Faber Agent: POST /connections/create-invitation , from the response get the invitation object (from { to })
- Bob Agent: POST /connections/receive-invitation with the invitation object
- Check both Faber Agent and Bob Agent by GET /connections and both are in **Active** status.

Here is the connections on all agents. Note that Faber now has two connections, one for Alice and one for Bob.

```
{
  "results": [
    {
      "created_at": "2019-12-09 06:35:59.862935Z",
      "their_label": "Alice Agent",
      "accept": "auto",
      "state": "active",
      "initiator": "self",
      "updated_at": "2019-12-09 06:36:34.361791Z",
      "invitation_mode": "once",
      "invitation_key": "3jHQM7DanUiMmhBf3Ee46WWwdS1e9SGAkvyjfrgbXn",
      "routing_state": "none",
      "connection_id": "1ed0ce65-9ed6-43ed-8836-fa2cdbcbdf27",
      "my_did": "LmkfMZneeYKAUTrcbsDvFK",
      "their_did": "FSDE7u344ZeNF2csCEiQph"
    },
    {
      "created_at": "2019-12-09 06:36:44.704139Z",
      "their_label": "Bob Agent",
      "accept": "auto",
      "state": "active",
      "initiator": "self",
      "updated_at": "2019-12-09 06:37:12.379336Z",
      "invitation_mode": "once",
      "invitation_key": "HF78x3MaMZ4K5Rn3LoifPHRovd9CqTSc4RDLJc7sonY4",
      "routing_state": "none",
      "connection_id": "798a16bc-c792-4414-887b-d6d5accad3b1",
      "my_did": "AyzaEuCQdQP9g4SMpGE19g",
      "their_did": "17wPecD73BB6x3JpjsrqZb"
    }
  ]
}
```

Connections seen in Faber Agent

```
{
  "results": [
    {
      "state": "active",
      "connection_id": "b65ccf45-elf9-4fca-a273-07bf6e3ba391",
      "routing_state": "none",
      "request_id": "a9524070-3732-40a3-95c3-f4c9d83d4ead",
      "invitation_key": "3jHQM7DanUiMmhBf3Ee46WWwdS1e9SGAkvyjfrgbXn",
      "invitation_mode": "once",
      "initiator": "external",
      "their_did": "LmkfMZneeYKAUTrcbsDvFK",
      "my_did": "FSDE7u344ZeNF2csCEiQph",
      "their_label": "Faber Agent",
      "accept": "auto",
      "updated_at": "2019-12-09 06:36:34.441158Z",
      "created_at": "2019-12-09 06:36:34.013235Z"
    }
  ]
}
```

Connection seen in Alice Agent

```
{
  "results": [
    {
      "invitation_mode": "once",
      "connection_id": "0d186c16-baa2-4371-9cbd-1e330d5d736f",
      "state": "active",
      "invitation_key": "HF78x3MaMZ4K5Rn3LoifPHRovd9CqTSc4RDLJc7sonY4",
      "updated_at": "2019-12-09 06:37:12.415205Z",
      "created_at": "2019-12-09 06:37:12.015025Z",
      "accept": "auto",
      "initiator": "external",
      "my_did": "17wPecD73BB6x3JpjsrqZb",
      "their_label": "Faber Agent",
      "their_did": "AyzaEuCQdQP9g4SMpGE19g",
      "routing_state": "none",
      "request_id": "96bf691b-ca03-4879-8c70-ef50891ca009"
    }
  ]
}
```

Connection seen in Bob Agent

For record purpose, here is the record of connection IDs, from Faber's perspective.

- Faber's connection ID to Alice: *1ed0ce65-9ed6-43ed-8836-fa2cdbcbdf27*
- Faber's connection ID to Bob: *798a16bc-c792-4414-887b-d6d5accad3b1*

We will use this in next step.

Step 2.2: Faber issue credentials to Alice and Bob, with proper individual data

The credential which Faber to create and issue requires the following three pieces of information

1. Connection ID (where Faber sends to)
2. Credential Definition ID (Faber created before)
3. Attributes for the credential receiver

The attributes for Alice is

- name: Alice Smith
- date: 2018-05-28
- degree: Maths
- average: 5

The attributes for Bob is

- name: Bob Johnson
- date: 2019-05-31
- degree: Physics
- average: 3

(In case you have not logged down, you can use `GET /connections` and get the Connection ID for Alice and for Bob. Then use `GET /credential-definitions/created` to obtain the Credential Definition ID.)

Here are the Credential to be sent to Alice and Bob. Replace the **Connection ID**, **Credential Definition ID**, and the **attributes** detail according to your own demo.

Credential to Alice

```
{
  "connection_id": "1ed0ce65-9ed6-43ed-8836-fa2cdbcbdf27",
  "cred_def_id": "VA3GvaAZRZwrH4dDtVWd6E:3:CL:8:default",
  "credential_proposal": {
    "@type": "did:sov:BzCbsNYhMrjHiqZDTUASHg;spec/issue-credential/1.0/credential-preview",
    "attributes": [
      {
        "name": "name",
        "value": "Alice Smith"
      },
      {
        "name": "date",
        "value": "2018-05-28"
      },
      {
        "name": "degree",
        "value": "Maths"
      },
      {
        "name": "average",
        "value": "5"
      }
    ]
  }
}
```

Credential to Bob

```
{
  "connection_id": "798a16bc-c792-4414-887b-d6d5accad3b1",
  "cred_def_id": "VA3GvaAZRZwrH4dDtVWd6E:3:CL:8:default",
  "credential_proposal": {
    "@type": "did:sov:BzCbsNYhMrjHiqZDTUASHg;spec/issue-credential/1.0/credential-preview",
    "attributes": [
      {
        "name": "name",
        "value": "Bob Johnson"
      },
      {
        "name": "date",
        "value": "2019-05-31"
      },
      {
        "name": "degree",
        "value": "Physics"
      },
      {
        "name": "average",
        "value": "3"
      }
    ]
  }
}
```

Faber Agent use `POST /issue-credential/send` to issue these two credentials.

POST

/issue-credential/send

Send credential, automating entire flow

Faber's Cloud Agent API

Parameters

Name

Description

body

(body)

Edit Value

Model

```
{
  "connection_id": "1ed0ce65-9ed6-43ed-8836-fa2cdcbcdf27",
  "cred_def_id": "VA3GvaAZRZwrH4dDtVWd6E:3:CL:8:default",
  "credential_proposal": {
    "@type": "did:sov:BzCbsNYhMrjHiqZDTUASHg:spec/issue-credential/1.0/credential-preview",
    "attributes": [
      {
        "name": "name",
        "value": "Alice Smith"
      },
      {
        "name": "date",
        "value": "2018-05-28"
      },
      {
        "name": "degree",
        "value": "Maths"
      }
    ]
  }
}
```

Cancel

Parameter content type

application/json

Execute

copy credential here

Faber uses this API to issue credential to Alice and Bob.

Step 2.3: Alice and Bob see credentials in their wallets

For both Alice and Bob Agent, use `GET /credentials` to get the credential received and stored in the wallet.

```
{
  "results": [
    {
      "referent": "b3159876-85b9-4alb-ac11-9dd5261c1f4c",
      "attrs": {
        "date": "2018-05-28",
        "average": "5",
        "degree": "Maths",
        "name": "Alice Smith"
      },
      "schema_id": "VA3GvaAZRZwrH4dDtVWd6E:2:degree schema:70.19.14",
      "cred_def_id": "VA3GvaAZRZwrH4dDtVWd6E:3:CL:8:default",
      "rev_reg_id": null,
      "cred_rev_id": null
    }
  ]
}
```

Credential in Alice's wallet

```
{
  "results": [
    {
      "referent": "aba2dlbb-f52e-4c09-821c-acc714d5f896",
      "attrs": {
        "degree": "Physics",
        "average": "3",
        "name": "Bob Johnson",
        "date": "2019-05-31"
      },
      "schema_id": "VA3GvaAZRZwrH4dDtVWd6E:2:degree schema:70.19.14",
      "cred_def_id": "VA3GvaAZRZwrH4dDtVWd6E:3:CL:8:default",
      "rev_reg_id": null,
      "cred_rev_id": null
    }
  ]
}
```

Credential in Bob's wallet

Step 3: Proof Presentation

Again, before any communication happens, a secured connection is established between two agents. In this demo, Acme first establishes one connection to Alice, and one to Bob. After that Acme issues a proof request to them, showing what type of proof is needed to qualify the job application. Alice and Bob will build the proof based on the credential in their own wallets. Finally Alice and Bob send the proof to Acme, and we will observe the result.

Step 3.1: Acme establishes connections to Alice and Bob

Similar process as Step 2.1. Here Acme agent first uses `POST /connections/create-invitation` to create invitation, and Alice uses `POST /connections/receive-invitation` with the invitation object. Repeat the same for connection between Acme and Bob.

Without showing the detail as above, for **record purpose** I just need the Connection IDs viewed from Acme to Alice and to Bob. They are used later for proof requests.

- Acme's connection ID to Alice: *ba1a486f-14ce-4835-a0b6-d42c6c7a2c3b*
- Acme's connection ID to Bob: *ab0577b7-d036-4b12-9303-26bd13f9d4f1*

Step 3.2: Acme sends Proof Request to Alice and Bob

Proof request details the requirement Acme is asking for. For our demonstration, Acme is asking for

- name, date and degree
- average is over 4
- all items following the Credential Definition specified by the ID

The JSON to be prepared to Alice and Bob. Note that the difference is on the **Connection ID** only. The Proof Request part is the same.

JSON for Proof Request to Alice

```
{
  "connection_id": "ba1a486f-14ce-4835-a0b6-d42c6c7a2c3b",
  "proof_request": {
    "name": "Proof of Education",
    "version": "1.0",
    "requested_attributes": {
      "0_name_uuid": {
        "name": "name",
        "restrictions": [
          {
            "cred_def_id": "VA3GvaAZRZwrH4dDtVWd6E:3:CL:8:default"
          }
        ]
      },
      "0_date_uuid": {
        "name": "date",
        "restrictions": [
          {
            "cred_def_id": "VA3GvaAZRZwrH4dDtVWd6E:3:CL:8:default"
          }
        ]
      },
      "0_degree_uuid": {
        "name": "degree",
        "restrictions": [
          {
            "cred_def_id": "VA3GvaAZRZwrH4dDtVWd6E:3:CL:8:default"
          }
        ]
      },
      "0_self_attested_thing_uuid": {
        "name": "self_attested_thing"
      }
    },
    "requested_predicates": {
      "0_average_GE_uuid": {
        "name": "average",
        "p_type": ">=",
        "p_value": 4,
        "restrictions": [
          {
            "cred_def_id": "VA3GvaAZRZwrH4dDtVWd6E:3:CL:8:default"
          }
        ]
      }
    }
  }
}
```

```
}
}
```

JSON for Proof Request to Bob

```
{
  "connection_id": "ab0577b7-d036-4b12-9303-26bd13f9d4f1",
  "proof_request": {
    "name": "Proof of Education",
    "version": "1.0",
    "requested_attributes": {
      "0_name_uuid": {
        "name": "name",
        "restrictions": [
          {
            "cred_def_id": "VA3GvaAZRZwrH4dDtVWd6E:3:CL:8:default"
          }
        ]
      },
      "0_date_uuid": {
        "name": "date",
        "restrictions": [
          {
            "cred_def_id": "VA3GvaAZRZwrH4dDtVWd6E:3:CL:8:default"
          }
        ]
      },
      "0_degree_uuid": {
        "name": "degree",
        "restrictions": [
          {
            "cred_def_id": "VA3GvaAZRZwrH4dDtVWd6E:3:CL:8:default"
          }
        ]
      },
      "0_self_attested_thing_uuid": {
        "name": "self_attested_thing"
      }
    },
    "requested_predicates": {
      "0_average_GE_uuid": {
        "name": "average",
        "p_type": ">=",
        "p_value": 4,
        "restrictions": [
          {
            "cred_def_id": "VA3GvaAZRZwrH4dDtVWd6E:3:CL:8:default"
          }
        ]
      }
    }
  }
}
```

```
}  
}
```

Acme uses `POST /present-proof/send-request` with the JSON object.

POST `/present-proof/send-request` Sends a free presentation request not bound to any proposal

Parameters **Acme's Cloud Agent API** Cancel

Name	Description
body (body)	Edit Value Model <pre>{ "connection_id": "8b0577b7-d036-4b12-9303-26bd11f9d6f1", "proof_request": { "name": "Proof of Education", "version": "1.0", "requested_attributes": { "0_name_uuid": { "name": "name", "restrictions": [{ "cred_def_id": "VA1GvaAZRZwrH4dDtVMd6E:3:CL:8:default" }] }, "0_date_uuid": { "name": "date", "restrictions": [{ "cred_def_id": "VA1GvaAZRZwrH4dDtVMd6E:3:CL:8:default" }] } } } }</pre> Cancel Parameter content type application/json

Execute Clear

copy proof request and paste here

Acme uses this API to send proof request to Alice and Bob.

We see from Alice's agent terminal and Bob's the process

- receiving the proof request
- checking credentials
- generating proof
- sending proof to Acme

```
Presentation: state = request_received , presentation_exchange_id = 3eda96df-fa35-43c8-aee3-ef210e960de9
#24 Query for credentials in the wallet that satisfy the proof request
#25 Generate the proof
#26 Send the proof to X
Presentation: state = presentation_sent , presentation_exchange_id = 3eda96df-fa35-43c8-aee3-ef210e960de9
```

Alice's Cloud Agent terminal

```
Presentation: state = request_received , presentation_exchange_id = 6d4dc0e7-4d6d-4776-a7b8-d35648462e19
#24 Query for credentials in the wallet that satisfy the proof request
#25 Generate the proof
#26 Send the proof to X
Presentation: state = presentation_sent , presentation_exchange_id = 6d4dc0e7-4d6d-4776-a7b8-d35648462e19
```

Bob's Cloud Agent terminal

Alice's and Bob's Agent terminals

From Acme, we can use `GET /present-proof/records` to see the proof sent by Alice and Bob. The `presentation_exchange_id` is the identifier of the presentation proof (local significant) and `state` will tell you the current status of this proof (presentation).

```
{
  "created_at": "2019-12-09 07:08:36.068662z",
  "presentation_exchange_id": "c3462ed5-0fff-44a8-8eda-830bcd4486aa",
  "initiator": "self",
  "connection_id": "bala486f-14ce-4835-a0b6-d42c6c7a2c3b",
  "role": "verifier",
  "state": "presentation_received",
  "presentation_request": {
    "name": "Proof of Education",
```

Presentation from Alice

```
{
  "created_at": "2019-12-09 07:09:19.035162z",
  "presentation_exchange_id": "31251183-7859-4ec0-9836-0c37dbfcbba",
  "initiator": "self",
  "connection_id": "ab0577b7-d036-4b12-9303-26bd13f9d4f1",
  "role": "verifier",
  "state": "presentation_received",
  "presentation_request": {
    "name": "Proof of Education",
```

Presentation from Bob

Acme sees proof presentations from Alice and Bob.

Currently the state is **presentation_received**.

From the response we see the presentation exchange ID for the requests.

- pres-ex-ID for Alice's proof: `c3462ed5-0fff-44a8-8eda-830bcd4486aa`
- pres-ex-ID for Bob's proof: `31251183-7859-4ec0-9836-0c37dbfcbba`

Step 3.3: Acme verifies the presentations received from Alice and Bob

Finally Acme use `POST /present-proof/{pres-ex-id}/verify-presentation` to see both Alice's and Bob's presentations.

The response tells the different results.

The screenshot displays a web interface with two sections. The top section, titled 'Code', shows a JSON response for Alice's proof verification. The response includes fields like 'created_at', 'presentation_exchange_id', 'initiator', 'connection_id', 'role', 'state', and 'presentation_request'. A yellow box highlights the text 'Alice's Proof is verified.' and a 'Download' button is present. The bottom section, titled 'Details', shows an error for Bob's proof verification. It displays a '500 Internal Server Error' with the message 'Server got itself in trouble' and a 'Download' button. Below the error message, the 'Response headers' are listed, including 'access-control-allow-credentials: true', 'access-control-allow-origin: http://localhost:8041', 'access-control-expose-headers:', 'connection: close', 'content-length: 55', 'content-type: text/plain; charset=utf-8', 'date: Mon, 09 Dec 2019 07:31:57 GMT', and 'server: Python/3.6 aiohttp/3.5.4'.

```
    },
    "created_at": "2019-12-09 07:08:36.068662Z",
    "presentation_exchange_id": "c3462ed5-0fff-44a8-8eda-830bcd4486aa",
    "initiator": "self",
    "connection_id": "bala486f-14ce-4835-a0b6-d42c6c7a2c3b",
    "role": "verifier",
    "state": "verified",
    "presentation_request": {
      "name": "Proof of Education"
    },
    "verified": "true"
  }
}
```

500 Internal Server Error
Server got itself in trouble

Response headers

```
access-control-allow-credentials: true
access-control-allow-origin: http://localhost:8041
access-control-expose-headers:
connection: close
content-length: 55
content-type: text/plain; charset=utf-8
date: Mon, 09 Dec 2019 07:31:57 GMT
server: Python/3.6 aiohttp/3.5.4
```

Alice passes the verification. Bob fails it.

For Alice, the response shows that the state is now changed to **verified**. And a new entry **verified: true** is seen. For Bob, since the predicate condition (average is over 4) is not met, the verification fails.

Overall Observation in this Demo

Here are some observations on Hyperledger Indy (ledger) and Hyperledger Aries (agent) from this demo.

1. In the ledger there is minimal information, good enough to establish the trust, including public DIDs, schemas and credential definitions. No personal data are found.
2. All personal data are communication over secured connections between agents.
3. There is no direct connection between credential issuers (Faber) and proof verifier (Acme). They trust each other, and other players in the same network.
4. To Alice and Bob, the credential issuance and proof presentation are two separate processes. They can receive the credentials and store them first. They can use them for proof later when needed.

5. When constructing a proof request, predicate attribute provides a way to prove something without disclosing the actual data. In the demo, the condition is “average is over 4”. It turns out Alice passes and Bob fails. In either case, the actual average (5 and 3 respectively) is not disclosed. This is handled by zero knowledge proof in the agent.

Hyperledger Indy

Hyperledger Aries

Aries Cloud Agent

Self Sovereign Identity

Identity



Follow

Written by KC Tam

2.5K followers · 313 following

Visit <http://www.ledgertech.biz/kcarticles.html> for all my works. Reach me on <https://www.linkedin.com/in/ktam1/> or follow me @kctheservant in Twitter.

Responses (7)



Jayathilakahansa

What are your thoughts?



shashank saxena

Dec 10, 2019



I receive an error while ./manage up.

The error says:

ERROR: pull access denied for von-network, repository does not exist or may require 'docker login': denied: requested access to the resource is denied



1



1 reply

[Reply](#)

Redcap Digital Solutions Pvt. Ltd.

Nov 20, 2022



Open in app ↗

Medium



Search



3



=> ERROR [12/13] RUN pip3 install --no-cache-dir -r demo/requirements.txt

→ demo git:(master) ./run_demo faber

Preparing agent image...

[+] Building 0.0s (16/17)

=> [internal]... [more](#)

[Reply](#)

Hakan Yildiz

Sep 4, 2020



Thanks for your article. It helped me to understand Aries ecosystem a bit better. Do you know how the verification works? As in: How can the verifier make sure that these credentials were given to Alice and Bob? Also, how can the verifier prove that these credentials haven't been copied multiple times?

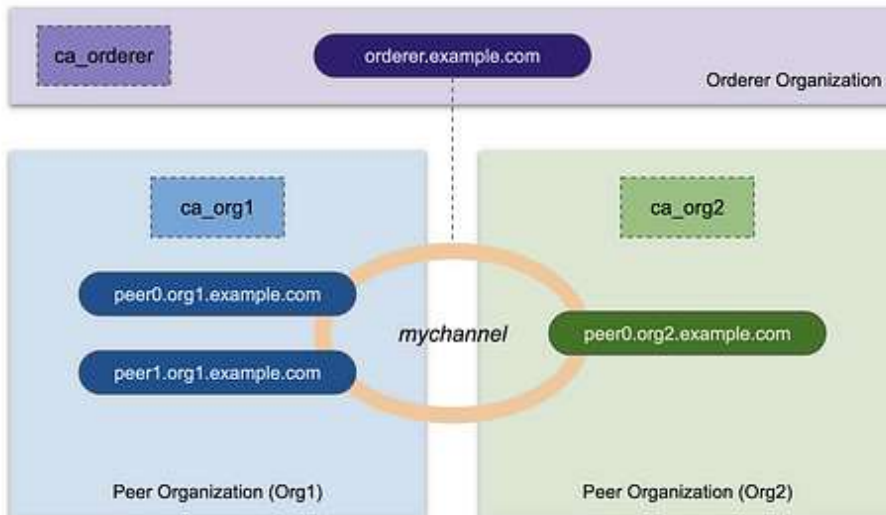


1 reply

[Reply](#)

See all responses

More from KC Tam



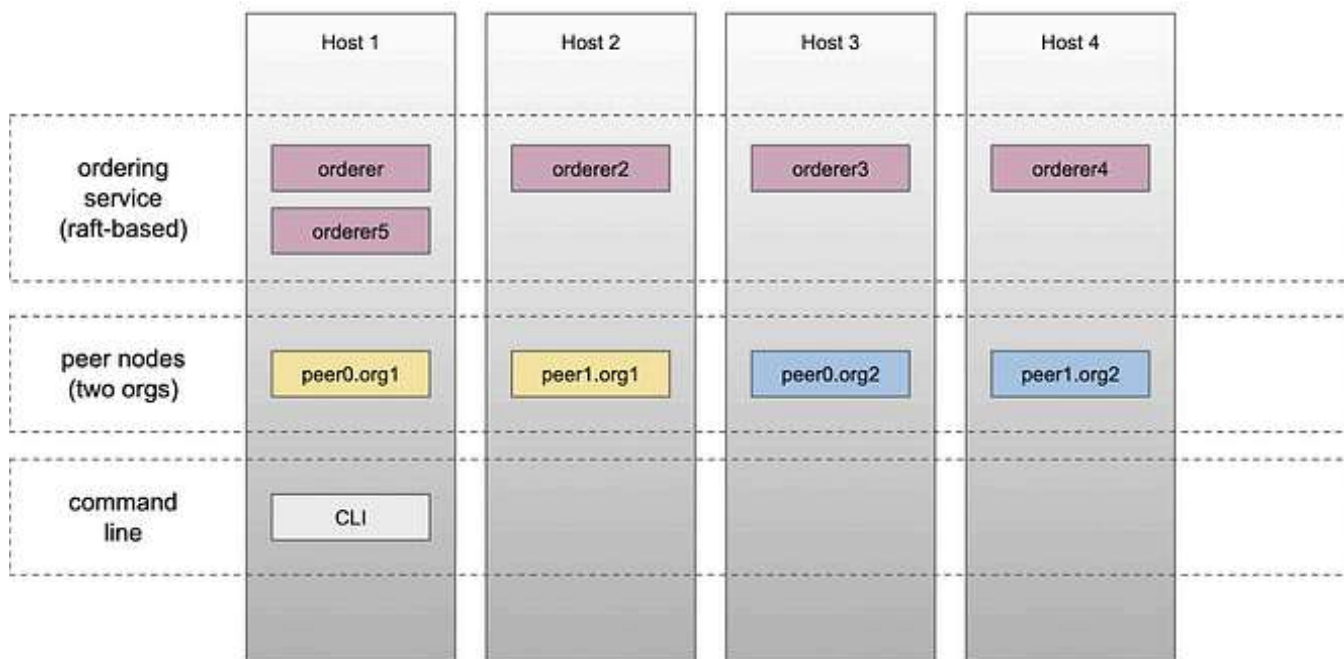
 KC Tam

Add a Peer to an Organization in Test Network (Hyperledger Fabric v2.2)

Overview

Aug 3, 2020  130  2



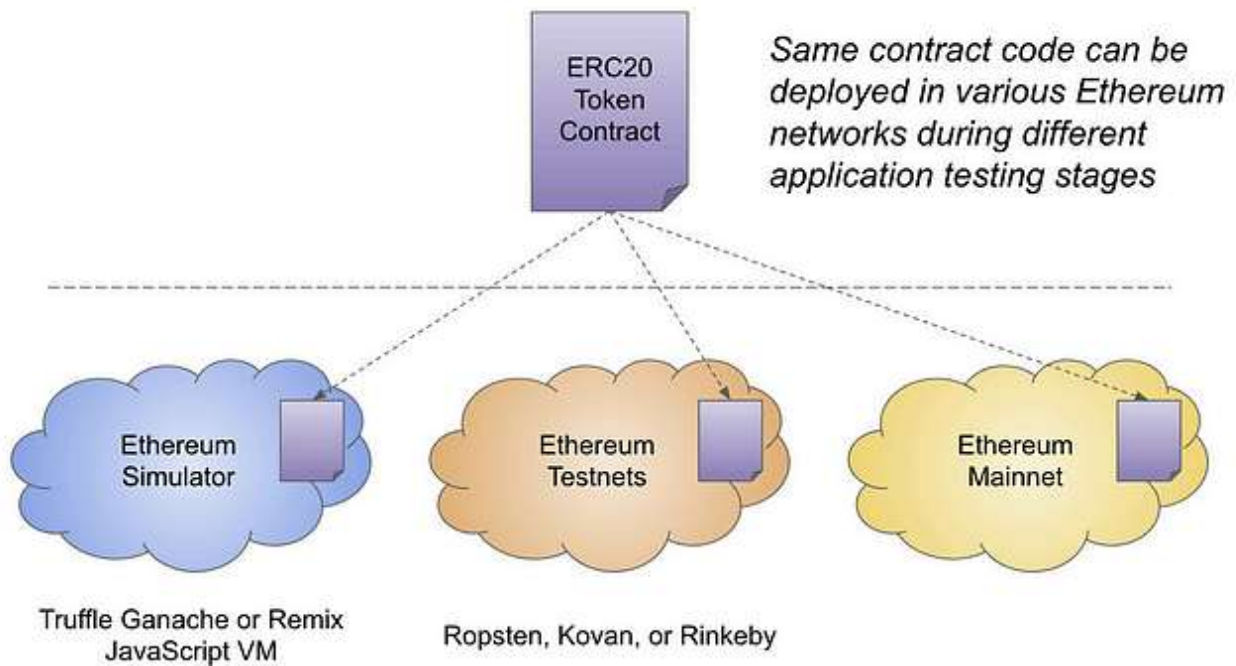
 KC Tam

Multi-Host Deployment for First Network (Hyperledger Fabric v2)

Overview

Aug 11, 2020  138  12

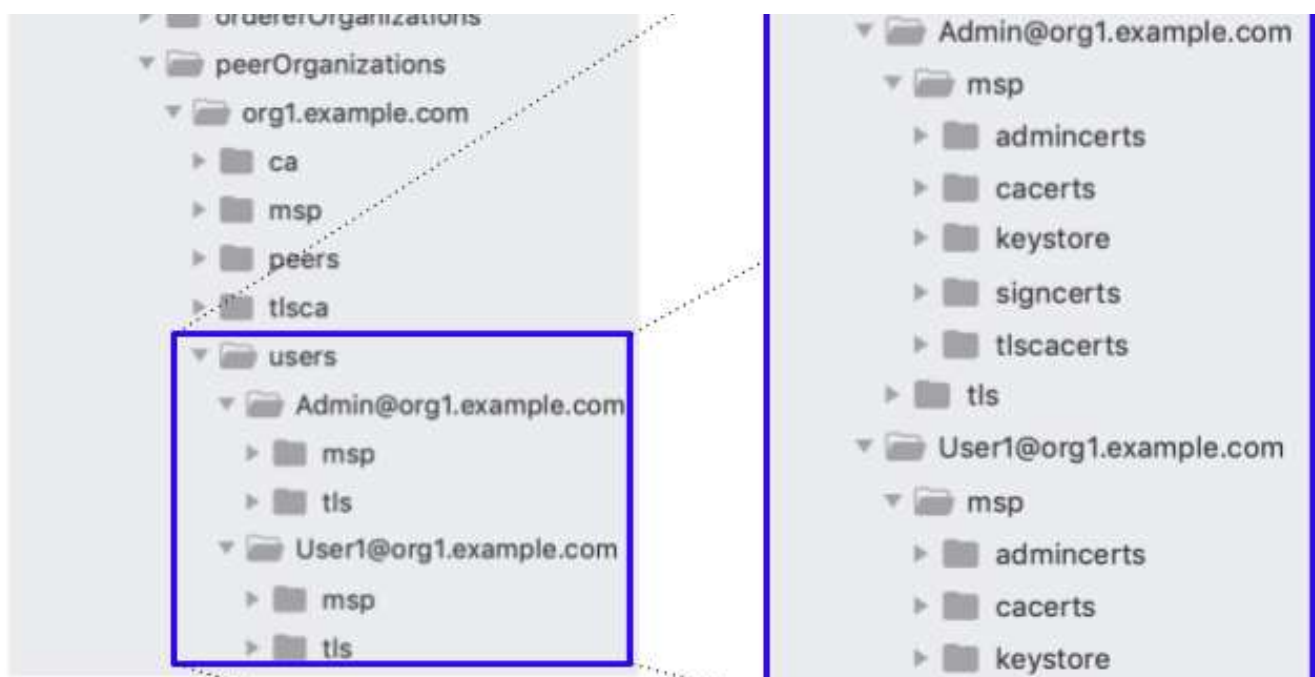


 KC Tam

Understanding Blockchain Frameworks: Ethereum and Hyperledger Fabric

A deeper look into them through infrastructure and application perspective.

Feb 2, 2019  221



 KC Tam

Identity in Hyperledger Fabric

Understanding Identity using Basic Network and Simple Asset Chaincode

Jun 12, 2019 🖱 268 💬 2

[See all from KC Tam](#)

Recommended from Medium



kubernetes

 Sohail Saifi

Kubernetes Is Dead: Why Tech Giants Are Secretly Moving to These 5 Orchestration Alternatives

I still remember that strange silence in the meeting room. Our CTO had just announced we were moving away from Kubernetes after two years...

★ Jun 7 🖱 3.4K 💬 140





 In Predict by iswarya writes

GPT-5 Is Coming in July 2025—And Everything Will Change

“It’s wild watching people use ChatGPT... knowing what’s coming.” —OpenAI insider

★ Jul 8 🖱 9K 💬 304





In LearnAltoprofit.com by Raj Monetix



I Built a Side Hustle with AI. Now It Pays Me \$800 every month

\$800/Month from My Laptop (No BS)



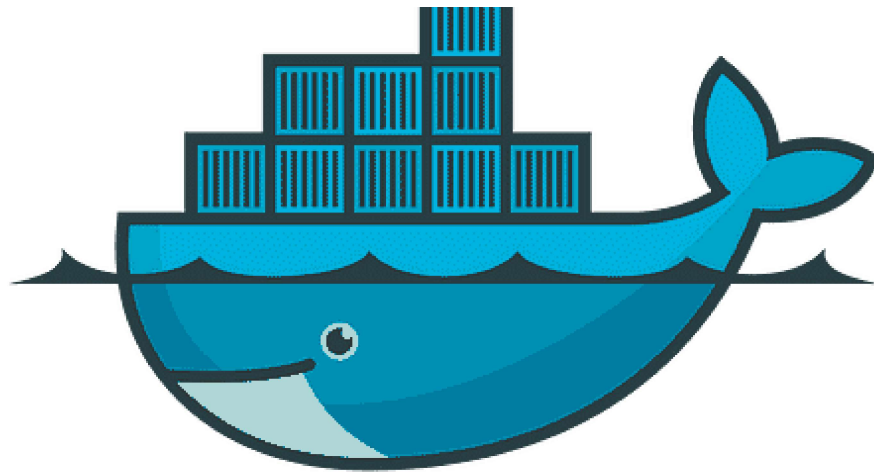
Jun 28



3.1K



118



docker



Abhinav

Docker Is Dead—And It's About Time

Docker changed the game when it launched in 2013, making containers accessible and turning “Dockerize it” into a developer catchphrase.

★ Jun 9 🤝 3K 💬 64

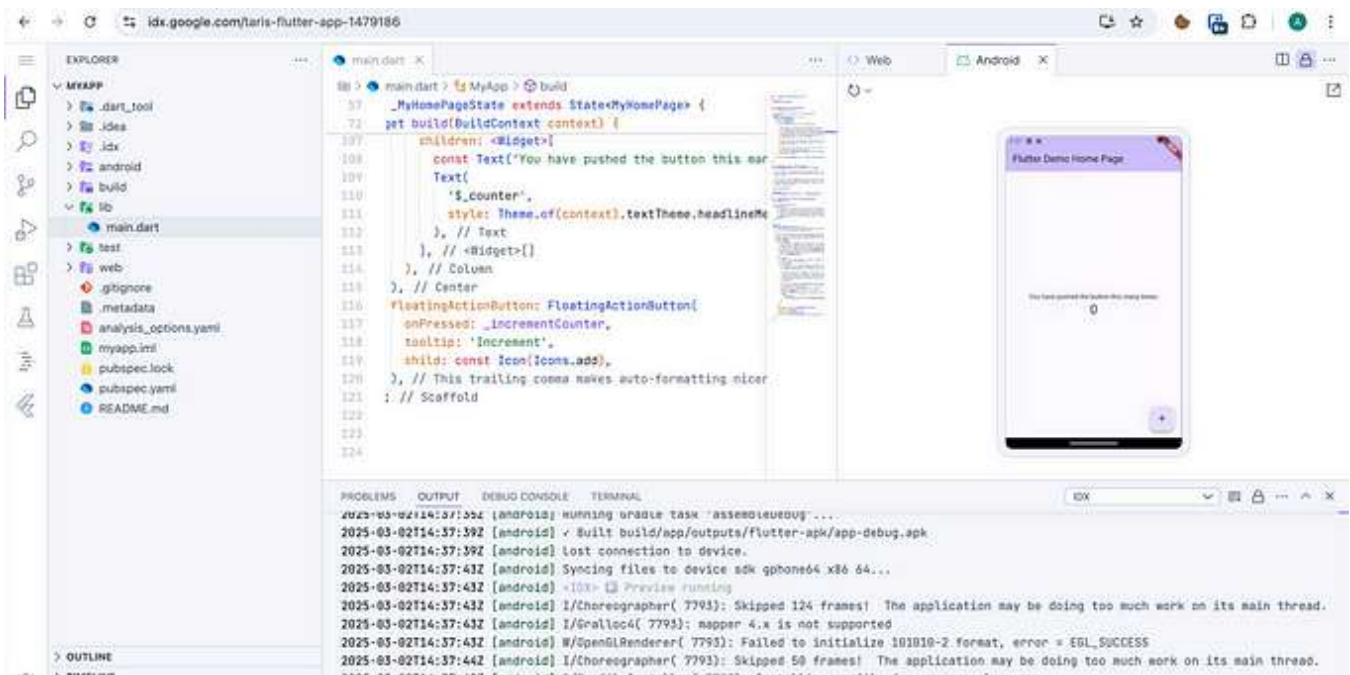


LONG In Long. Sweet. Valuable. by Ossai Chinedum

I'll Instantly Know You Used Chat Gpt If I See This

Trust me you're not as slick as you think

★ May 16 🤝 17.1K 💬 985



CB In Coding Beauty by Tari Ibaba

This new IDE from Google is an absolute game changer

This new IDE from Google is seriously revolutionary.



Mar 12



6.1K



367



See more recommendations